

Redes II

Práctica 1

Jorge Jiménez Oropesa, Antonio Moroño Moreno

Grupo 2301, Pareja 14

En esta memoria tratamos el desarrollo y funcionalidad del servidor HTTP que hemos desarrollado conjuntamente para la primera práctica de esta asignatura.

Paralelización

Con objeto de poder atender a numerosos clientes simultáneamente, nuestro servidor utiliza **multithreading reactivo**. Utilizando la librería *pthread*, crea un hilo cada vez que recibe una conexión, existiendo un número máximo de hilos que pueden estar trabajando simultáneamente y que queda determinado por la variable 'max_clients' configurable desde 'server.conf'.

Elegimos utilizar hilos y no procesos por dos motivos. Principalmente, **los hilos resultan significativamente más eficientes, e introducen un menor overhead**, lo que es de vital importancia para garantizar un desempeño adecuado del servidor. Adicionalmente, encontramos la gestión de los hilos más sencilla que la de procesos, así que no pareciendo haber desventajas de otro tipo, resulta razonable elegir la opción con la que más cómodos nos sentíamos.

Por otro lado, el servidor es, como hemos mencionado, **reactivo**. Elegimos esta filosofía frente a un pool porque consideramos que así **el servidor puede ajustarse mejor a la demanda**. Con un pool de hilos, si el servidor apenas recibe tráfico en algún momento, seguiremos teniendo creados hilos que no estamos utilizando. El reactivo en cambio es dinámico, lo que mejora el rendimiento. Además, estableciendo un número máximo de clientes, garantizamos que ante una demanda completa, podemos ofrecer un desempeño similar al de un pool con ese tamaño. La principal desventaja del reactivo es que resulta más complicado de implementar y gestionar, pero estábamos dispuestos a dedicarle más esfuerzo si con ello obtenemos un mejor resultado.

Para gestionar los hilos hemos creado un script, '**thread_manager.c**', junto a su correspondiente '.h'. Básicamente almacena en memoria un array de tamaño 'max_clients', inicializado al iniciar el servidor, y en el que se **almacenan los ids de los hilos activos**. Cuando se crea un hilo su id se guarda en el array, y cuando se cierra la conexión, se elimina del mismo. Gracias a un contador y control mediante semáforos, evitamos las condiciones de carrera y garantizamos que el número de hilos activos nunca exceda el máximo de clientes. Esta implementación nos permite que al mandar **SIG_INT** (o, realmente cualquier otra señal) al hilo principal, **este sea capaz de reenviarla al resto de hilos activos** con *pthread_kill*. Gracias a esto podemos cerrar el programa evitando pérdidas de memoria y garantizando la correcta finalización de los hilos, de modo que el principal espera al resto para terminar.

HTTP

Para el parseo de HTTP, hemos decidido crear **nuestras propias funciones**. Estas están integradas junto al resto de código para la gestión del protocolo HTTP (respuestas, ejecución de scripts, ...) en '**http_manager.c**' (acompañado del correspondiente '.h'). Hemos

decidido que lo más conveniente era crear una función global para todos los verbos ya que los tres verbos comparten muchas de las acciones a realizar.

La manera en que se procesan las peticiones http en nuestro servidor es el siguiente: Tras recibir un mensaje el servidor llama a la función 'http_parse_ppetition' la cual presea las cabeceras y en caso de estar bien formulada la petición llama a la función interna 'http_process' indicando el verbo de la petición. Para todos los verbos esta función verifica el archivo referenciado en el path y realiza la acción asociada al verbo de la petición. Dentro de esa acción se genera una respuesta que es lo que devuelve a 'server.c' un struct con la respuesta http. Después se envía y se libera la memoria asociada.

Log

Para el logueo de mensajes **hemos creado una librería sencilla**. Esta establece tres niveles de mensaje: **INFO, WARNING y ERROR**, todos ellos autoexplicativos. Cuando se quiere mostrar algún mensaje en el log, basta con importar la librería y llamar a la función log_message, que no sólo muestra el mensaje y su nivel, sino que **añade información relevante como la hora y el thread_id** del hilo que lo ha llamado. Los logs se guardan en el archivo 'server.log'.

Funcionales adicionales

Hemos implementado algunas funciones adicionales que no eran obligatorias:

- 1) **Ataque DOS**. Nuestro servidor dispone de un "botón secreto" que, al ser ejecutado, ejecuta un script de python. Este script es capaz de detectar el puerto del servidor accediendo a 'server.conf' e iniciar un ataque *dos* que lanza un gran número de conexiones volátiles al servidor. **En caso de que el servidor sea capaz de gestionar correctamente el ataque** (y así ha sido en nuestras pruebas), **el script pinte un pequeño dibujo ASCII** de un demonio y el mensaje de que se ha ejecutado el ataque DOS. Este mensaje se muestra en el navegador gracias a la redirección de la salida de los scripts ejecutados.
- 2) **Parámetros personalizables GET y POST en python**. Hemos modificado el archivo index.html de modo que el usuario puede introducir por teclado los parámetros que se pasaran al script de python tanto con el verbo GET como con el verbo POST.